



Universidad de Jaén

Bloque 3

Ingeniería de Requisitos



EPS
Escuela Politécnica
Superior de Jaén

Tema 7. Modelado de los Objetos

L. Alfonso Ureña López Eugenio Martínez Cámara

Departamento de Informática
Universidad de Jaén

Fundamentos de Ingeniería del Software
Grado en Ingeniería Informática
2º Curso

Contenidos

- 1 Objetivos
- 2 Representación del objeto
 - El objeto
 - La abstracción
 - Clases de objetos
 - Encapsulación
- 3 Relaciones con otros objetos
 - Especialización y generalización
 - Herencia
 - Clases abstractas y concretas
 - Polimorfismo
 - Composición
 - La especialización de los elementos. La noción de estereotipo en UML
- 4 UML: representación de asociaciones entre clases
 - Asociación entre clases
 - Navegación
 - Asociación reflexiva
 - Clases-asociaciones
 - Relaciones entre asociaciones

Objetivos

- Conocer y comprender los conceptos de **objeto**, **abstracción**, **clases de objetos** o **encapsulación**
- Conocer y diferenciar los conceptos de **herencia** (especialización y generalización), **polimorfismo** y **composición**
- Saber diferenciar entre **composición fuerte** y **composición débil**
- Conocer y diferenciar los conceptos de **clase abstracta** y **clase concreta**
- Conocer y representar el concepto de **estereotipo**
- Conocer los distintos **tipos de asociaciones entre clases**
- Conocer las **peculiaridades de UML 2** para la representación de objetos, clases y sus relaciones

El objeto

- Un objeto es una **entidad identificable** del mundo real. Puede tener una existencia física (un caballo, un libro) o no (una sesión de una conferencia). **Identificable** significa que el objeto se puede designar
- En UML todo **objeto** posee un conjunto de **atributos** (estructura) y un conjunto de **métodos** (comportamiento)
- Incluso los **objetos estáticos** del mundo real son percibidos en **UML** como **dinámicos**. Así, en UML, un libro es concebido como un objeto capaz de abrirse en una página determinada
- Todo **sistema** concebido en **UML** está compuesto por **objetos** que **interactúan entre sí** y realizan operaciones propias de su **comportamiento**

La abstracción

- La **abstracción** es un principio muy importante en modelado. Consiste en tener en cuenta únicamente las **propiedades pertinentes** de un **objeto** para un **problema concreto**
- Los **objetos** utilizados en **UML** son **abstracciones del mundo real**

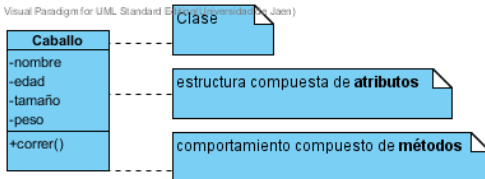
Clases de objetos



Clase de objetos

Se define como un conjunto de **objetos similares**, con la **misma estructura** y **comportamiento**, y constituidos por los **mismos atributos** y **métodos**

Visual Paradigm for UML Standard Edition (Universidad de Jaén)

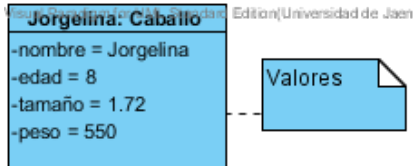


Clases de objetos



Instancia de clase

Cada **objeto** de una clase se denomina **instancia de clase** y se distinguen por tener una **identidad propia** y un comportamiento específico a través de sus atributos



Encapsulación

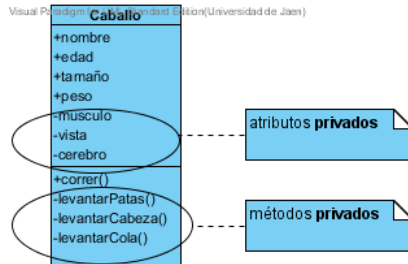
- La **encapsulación** consiste en **ocultar** los **atributos** y **métodos** del objeto a otros objetos
- Algunos atributos y métodos tienen como único objeto **tratamientos internos** del objeto y no deben estar expuestos a otros objetos. Una vez encapsulados, pasan a denominarse **atributos** y **métodos privados** del objeto
- La **encapsulación** es una **abstracción**, ya que **simplifica** la representación del **objeto** con respecto al resto de objetos. Esta representación simplificada está representada por los **atributos** y **métodos públicos** del objeto

Encapsulación

En notación **UML** los atributos y métodos irán precedidos por un signo según su tipo:

Tipo	Signo	Descripción
público	+	Elemento no encapsulado visible para todos
protegido	#	Elemento encapsulado visible en las subclases de la clase
privado	-	Elemento encapsulado visible sólo en la clase
empaquetado	~	Elemento encapsulado visible sólo en las clases del mismo empaquetado

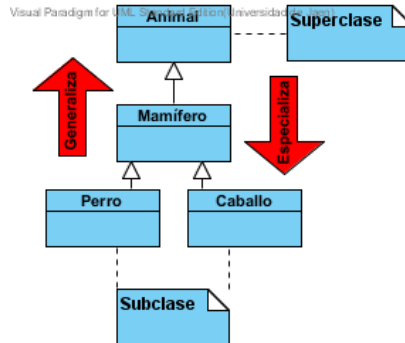
Encapsulación



Especialización y generalización

- Las **clases** pueden especificarse como **subconjuntos** de **otras** clases siempre que éstos constituyan conjuntos de objetos **similares**
- La **generalización** es la relación **inversa** a la **especialización**. Si una clase es una especialización de otra clase, esta última es una generalización de la primera
- La clase **especializada** es una **subclase** de la **generalizada**, la cual a su vez es una **superclase** de la especializada

Especialización y generalización



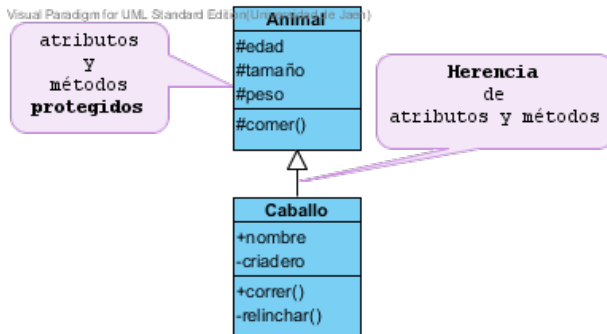
Herencia

- La **herencia** es la propiedad que hace que una **subclase** se beneficie de la **estructura** y **comportamiento** de su **superclase**
- Las **instancias** de una **subclase** son también **instancias** de la **superclase**, por lo que también se **benefician** de la **estructura** y **comportamiento** definidos por la **superclase**

Objetivos
Representación del objeto
Relaciones con otros objetos
UML: representación de asociaciones entre clases

Especialización y generalización
Herencia
Clases abstractas y concretas
Polimorfismo
Composición
La noción de estereotipo en UML

Herencia



Clases abstractas y concretas



Clase abstracta

Es aquella que tiene **subclases asociadas**, y por tanto **no** poseen directamente **instancias**, al pertenecer éstas a las subclases declaradas

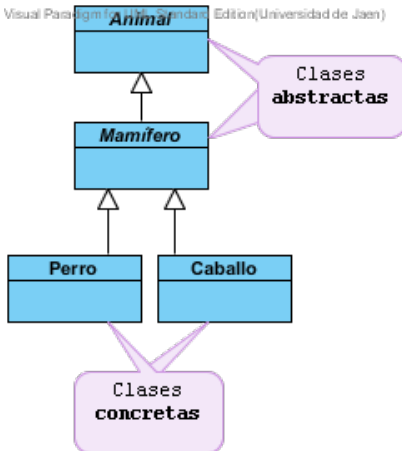


Clase concreta

Es aquella que **sí permite instancias** de esa clase y no poseen subclases asociadas

Clases abstractas y concretas

Visual Paradigm for UML Standard Edition (Universidad de Jaén)



Polimorfismo



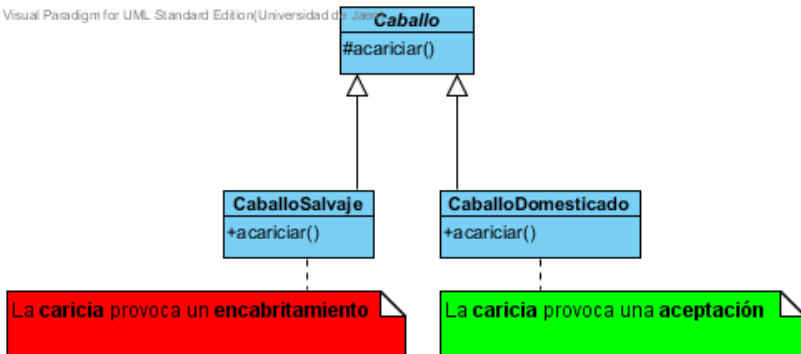
El **polimorfismo** significa que una **clase** (generalmente **abstracta**) representa un conjunto formado por **objetos diferentes**, ya que éstos son **instancias** de **subclases diferentes**. Cuando se llama a un **método** del mismo nombre esto se traduce en **comportamientos diferentes**.

Objetivos
Representación del objeto
Relaciones con otros objetos
UML: representación de asociaciones entre clases

Especialización y generalización
Herencia
Clases abstractas y concretas
Polimorfismo
Composición
La noción de estereotipo en UML

Polimorfismo

Visual Paradigm for UML Standard Edition (Universidad de Jaén)

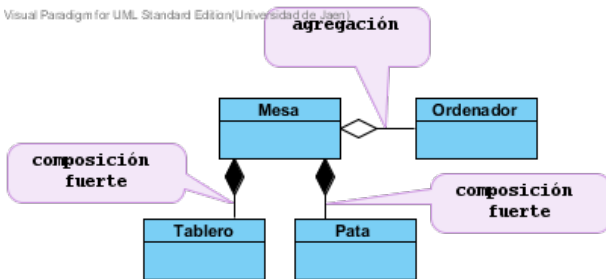


Composición

- Un **objeto** puede ser complejo y estar **compuesto** por otros objetos. La **asociación** que une a estos objetos es la **composición**, que se define a nivel de sus clases, pero cuyos vínculos se establecen entre las instancias de sus clases
- La composición puede adoptar dos formas:
 - **Composición débil** o **agregación**: los **componentes** pueden ser **compartidos** por varios objetos complejos
 - **Composición fuerte**: los **componentes no pueden compartirse** y la destrucción del objeto compuesto conlleva la destrucción de sus componentes

Composición

Visual Paradigm for UML Standard Edition (Universidad de Jaén)

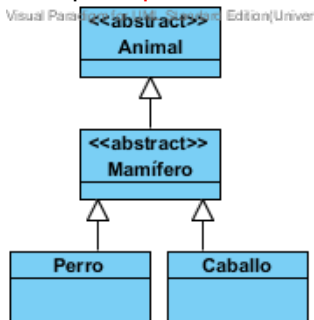


La noción de estereotipo en UML

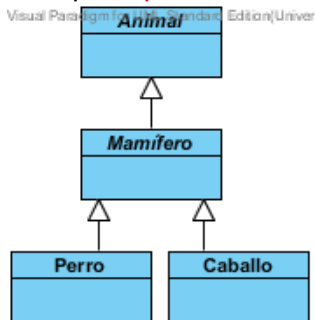
- El concepto de clase abstracta es un concepto especializado de clase. En **UML** los **estereotipos** pueden ser:
 - **Implícitos**: se escribe el **nombre** de la clase abstracta en **cursiva**
 - **Explícitos**: podemos prescindir de poner el nombre de la clase en cursiva precisando de forma explícita el **estereotipo «abstract»**

La noción de estereotipo en UML

Estereotipo **explícito** «abstract»



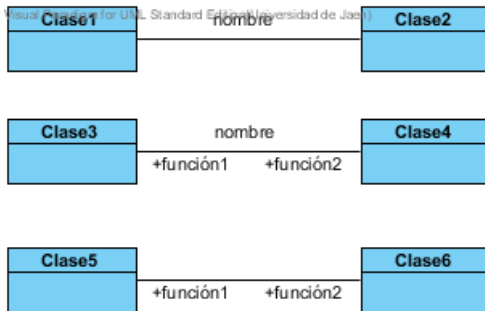
Estereotipo **implícito** «abstract»



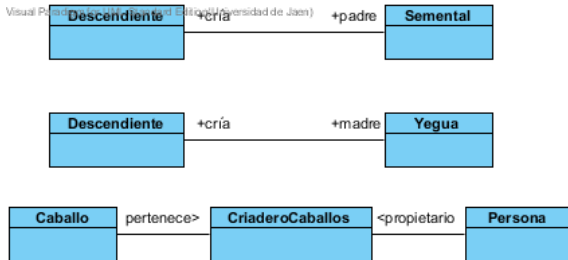
Asociación binaria entre clases

- El **nombre** de la **asociación** se indica encima de la **línea**. Para indicar el **sentido** de lectura del nombre de la asociación puede precederse del **signo <** o seguirse del **signo >**
- Los **extremos** de la asociación también pueden recibir un **nombre** representativo de la **función** que desempeñan en la asociación las **instancias** de la clase correspondiente
- Dicha **función** se representaría como un **atributo** de la **clase** situada al **otro extremo** de la asociación y cuyo **tipo** sería la **clase** situada en el **mismo extremo** donde aparece la función

Asociación binaria entre clases



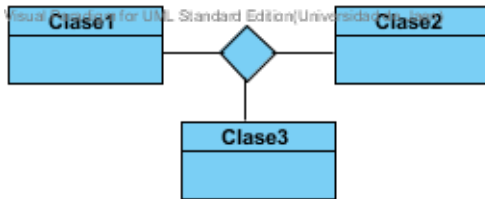
Asociación binaria entre clases



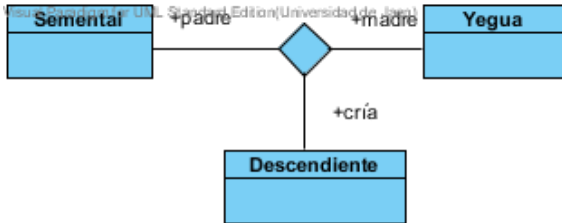
Asociación ternaria entre clases



La representación gráfica de una asociación ternaria y superiores consiste en un rombo que une las diferentes clases



Asociación ternaria entre clases

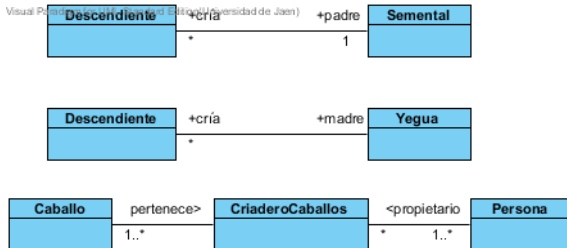


Cardinalidad de las asociaciones

- La **cardinalidad** situada en un **extremo** de una **asociación** indica a cuántas **instancias** de la clase situada en ese **mismo extremo** está **vinculada** una **instancia** situada en el **extremo opuesto**
- Si **no** hay especificación **explícita** ésta se considera **1**

Especificación	Cardinalidad
0..1	cero o una vez
1	únicamente una vez
*	de cero a varias veces
1..*	de una a varias veces
M..N	entre M y N veces
N	N veces

Ejemplo de cardinalidad



Navegación

- Por defecto, las **asociaciones** tienen una **navegación bidireccional**; es decir, es posible determinar los vínculos de la asociación desde una instancia de cada una de las clases implicadas
- La navegación **bidireccional** suele ser más compleja de realizar para los desarrolladores, por lo que **conviene evitarlas** en la medida de lo posible
- Para especificar un **único sentido** de **navegación** se dibuja la asociación en forma de **flecha**

Objetivos
Representación del objeto
Relaciones con otros objetos
UML: representación de asociaciones entre clases

Asociación entre clases
Navegación
Asociación reflexiva
Clases-asociaciones
Relaciones entre asociaciones

Ejemplo navegación

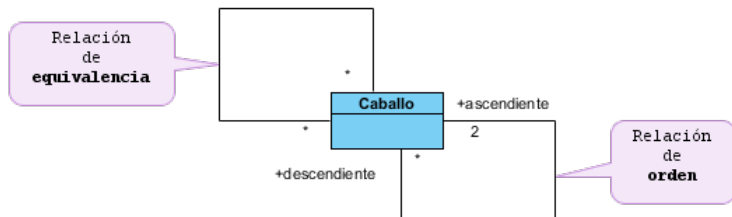


Asociación reflexiva

- Se trata de **asociaciones** en las cuales encontramos en los dos extremos de una relación a una **misma clase**
- Resulta **recomendable** asignar un **nombre** a la **función** desempeñada por la clase en **cada extremo** de la asociación
- Sirven para describir dentro del conjunto de instancias de una clase:
 - Grupos de instancias: **relación de equivalencia**
 - Jerarquía dentro de las instancias: **relación de orden**

Ejemplo asociación reflexiva

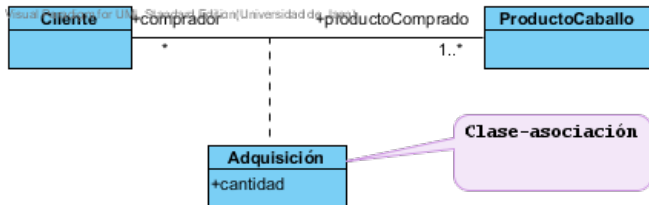
Visual Paradigm for UML Standard Edition (Universidad de Jaén)



Clases-asociaciones

- Un **vínculo** entre instancias de clases puede llevar **información** específica a dicho vínculo
- En estos casos, la **asociación** que describe el **vínculo** se comporta como una **clase** y sus instancias son elementos de la asociación
- Recibe el nombre de **clase-asociación** y se comportan como cualquier otra clase

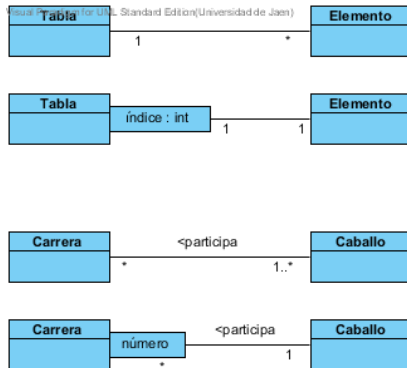
Ejemplo clase-asociación



La calificación de una asociación

- En caso de **cardinalidad máxima no finita** en un **extremo** de la **asociación**, si las instancias situadas en dicho extremo son calificables, la **calificación** permite **acotar superiormente** la **cardinalidad** asociada
- La **calificación** de una instancia es un **valor** o conjunto de valores que **permiten encontrar** dicha **instancia**
- Suelen ser **índices** para encontrar un elemento en una **tabla**, o **llaves** para, por ejemplo, localizar una línea en una base de datos relacional

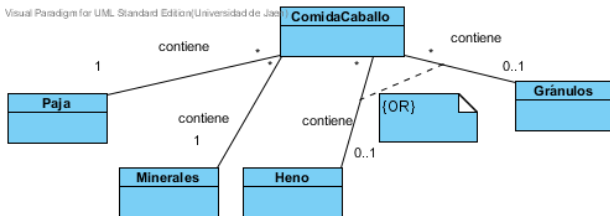
Ejemplo de calificación de una asociación



La relación de exclusión



En ocasiones, determinadas asociaciones no pueden darse a la vez, por lo que UML permite establecer dicha **restricción** enlazando las **asociaciones** implicadas mediante **líneas discontinuas** e indicando mediante una **nota** la restricción existente



La comida puede contener heno o gránulos, pero no los dos a la vez

La relación de orden



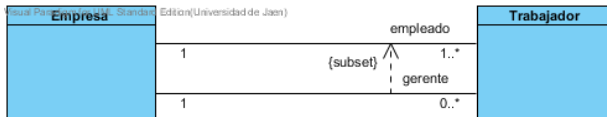
En ocasiones, las posibles instancias de la clase de uno de los extremos de la asociación tienen que mantener un **orden** en su **relación** con las **instancias** de la **clase** del otro extremo, por lo que UML permite establecer dicha **restricción de orden** mediante el uso del término **{ordered}** en el lado de la asociación donde hay que mantener dicho orden



La relación de inclusión



Pueden presentarse dos o más **asociaciones entre dos clases**, de manera que las **instancias** de la **clase** de uno de los **extremos** de una de las **asociaciones** están **incluidas** dentro de las **instancias** de la **clase** de ese **mismo extremo** de **otra asociación**, por lo que UML permite establecer dicha **restricción de inclusión** mediante una asociación discontinua entre los extremos de las asociaciones relacionadas, terminada en flecha en el sentido de la asociación contenedora. La relación de inclusión se etiqueta con el término **{subset}**



Bibliografía



L. Debrauwer and F. Van der Heyde

UML 2. Iniciación, Ejemplos y Ejercicios Corregidos. 3ª Ed.
Ediciones ENI, 2013

Enlaces de interés



<http://www.uml-diagrams.org/class-diagrams.html>

UML Class Diagrams Notation



<http://www.uml-diagrams.org/class-diagrams-overview.html>

UML Class Diagrams Overview